

MVSS: Blockchain Cross-shard Account Migration Based on Multi-version State Synchronization

1st Weihang Wang

Tianjin International Engineering Institute College of Intelligence and Computing College of Intelligence and Computing
Tianjin University Tianjin University
Tianjin, China Tianjin, China
weihan_wang@tju.edu.cn xiulong_liu@tju.edu.cn mly@tju.edu.cn

2nd Xiulong Liu3rd Liyuan Ma

4th Hao Xu
College of Intelligence
and Computing
Tianjin University
Tianjin, China
hao_xu@tju.edu.cn

5th Gaowei Shi
College of Intelligence
and Computing
Tianjin University
Tianjin, China
shigaowei@tju.edu.cn

6th Juncheng Ma
College of Intelligence
and Computing
Tianjin University
Tianjin, China
juncheng@tju.edu.cn

7th Keqiu Li (*IEEE Fellow*)
College of Intelligence
and Computing
Tianjin University
Tianjin, China
keqiu@tju.edu.cn

Abstract—The cross-shard account migration mechanism is one of the most advanced solutions to the unbalanced load problem in blockchain sharding systems. Existing solutions employ locking schemes to ensure the atomicity of transactions and the consistency of account states. However, these methods suffer from high transaction locking rates, degraded system performance, and disordered transaction execution sequences. To overcome these limitations, this paper proposes the first cross-shard account migration mechanism based on Multi-Version State Synchronization (MVSS), which ensures sequential execution of transactions while reducing the locking rate and latency, thereby enhancing system throughput. Specifically, MVSS divides related transactions into two categories based on their arrival time, and innovates to have the source shard and the destination shard process each category of transactions, instead of blocking all transactions at the source shard as in the existing schemes. Furthermore, in the special timestamp interleaving scenario, MVSS synchronizes the multiple version states of the account among the related shards according to the fine-grained order, which effectively maintains the consistency of the transaction sequence and the account state. Additionally, this paper delves into the security aspects of MVSS, proposing effective solutions to combat double-spending and replay attacks. Finally, We conducted extensive experiments based on a blockchain sharding simulator using over 2000 lines of *Go* to evaluate the performance of MVSS. The experimental results indicate that, compared to the state-of-the-art solutions such as Fine-tuned scheme, MVSS reduces locking time by 23.4%, decreases transaction latency by 7.5%, and improves throughput by 2.3%.

Index Terms—Blockchain, sharding, account migration

I. INTRODUCTION

A. Background and Motivation

The scalability issue of blockchain has emerged as a significant bottleneck to enhancing its performance. Sharding technology [1]-[8] addresses this challenge by dividing the nodes of the entire blockchain network into multiple autonomous committees, each processing transactions in parallel to improve system throughput. This approach has gained

recognition as an effective scalability solution. Solutions such as Brokerchain [9] use Metis algorithms that optimize account distribution across shards, balancing the load between shards and minimizing the number of cross-shard transactions. Building upon these algorithms, cross-shard account migration schemes [13]-[15] reallocates accounts across shards without stopping the process of regular transactions.

However, existing cross-shard account migration schemes typically lock more than 50% transactions related to the migrating account in the source shard to ensure consistency of account states. This practice increases transaction latency, limits the concurrent processing capability of the sharding system, and ultimately decreases system throughput. More critically, locking cross-shard transactions can lead to transaction disorder issues. For instance, consider three transactions, TX_1 , TX_2 and TX_3 , respectively, receive 5 tokens, 3 tokens and withdraw 10 tokens from an account that initially holds 2 tokens. If we execute these transactions in the order of TX_1 - TX_2 - TX_3 , all submissions succeed, resulting in a final account balance of 0. Conversely, if TX_1 and TX_3 are processed by the source shard while TX_2 is handled by the target shard after locking, and we execute in the order TX_1 - TX_3 - TX_2 , TX_3 would fail due to insufficient balance, leaving a final account balance of 10 tokens. This example illustrates that transaction disorder not only adversely affects the outcomes of transactions but also disrupts the data consistency of the blockchain system.

In summary, existing solutions have not effectively mitigated the issues of transaction locking and disorder in cross-shard account migration. Therefore, this paper presents a solution based on the multi-version state synchronization, which coordinates the sequential processing of related transactions by the source shard and the target shard. This approach aims to reduce both the proportion and latency of locked transactions while improving system throughput during cross-shard account

TABLE I
PERFORMANCE OF CROSS-SHARD ACCOUNT MIGRATION
PROTOCOL

Protocol	Consensus round	TX_{payer} Locking	TX_{payee} Locking	TXs disorder
Full-locked	2	Yes	Yes	Yes
Semi-locked	4	No	Yes	Yes
MVSS	4	No	No	No

migration. It is also important that related transactions are executed in order.

B. Limitations of Prior Art

Currently, there are two types of cross-shard account migration schemes: full-locked and semi-locked. The full-locked scheme, exemplified by the LB mechanism [14], locks all relevant transactions during the account migration process, transferring them along with the account to the target shard. Only after the target shard completes the account reconstruction are these transactions processed. While this method ensures the atomicity of transactions and consistency of account states during migration, the complete locking of transactions throughout the process significantly increases transaction latency. Moreover, during bulk account migrations, extensive locking can hinder the concurrent processing capabilities of the sharding system, leading to a decline in overall throughput.

In contrast, the semi-locked mechanism, represented by Fine-tuned [15], only locks the transactions that are deducted from the account (TX_{payer}) at the start of the migration. Transactions that deposit into the account (TX_{payee}) remain unlocked and are processed directly by the source shard, updating the account state. However, since the semi-locked scheme requires state updates on the account in the target shard, it incurs more consensus rounds than the full-locked scheme. Thus, while it reduces the proportion of locked transactions, the locking time is extended, potentially causing some transactions to fail due to timeouts or dependencies on other transactions, ultimately impacting the overall performance of the system. Furthermore, existing schemes have not addressed the issue of transaction disorder during cross-shard account migration, leaving the consistency unguaranteed.

C. MVSS in a Nutshell

To overcome the limitations of existing solutions, this paper proposes MVSS, a cross-shard account migration mechanism based on multi-version state synchronization, which significantly reduces the locked transactions ratio. Specifically, to alleviate transaction locking issues, MVSS innovatively schedules the source shard and the target shard to process related transactions cooperatively, rather than having the source shard process part of the transactions or lock them completely. This significantly lowers the locking time of transactions. Additionally, to address the problem of transaction disorder, MVSS implements fine-grained ordering and multi-version state synchronization between the source and target shards, facilitating synchronization of account states during the ordered

processing of related transactions. This establishes a bridge for cross-shard transaction processing. Furthermore, MVSS protects against two types of attacks that may arise during the cross-shard account migration period, ensuring system security.

D. Challenges and Solutions

When implementing the MVSS mechanism, we need to address the following key technical challenges:

The first challenge is how to efficiently schedule transaction cooperating to process transactions between the source and target shards, thereby reducing transaction locking. Since the isolation of transaction processing within each shard, coupled with a lack of information sharing between shards, processing multiple transactions for the same account in parallel across shards is difficult. Traditional optimistic concurrency control methods can lead to transaction conflicts and rollbacks, increasing system overhead and jeopardizing the consistency of system data. To tackle these issues, this paper first clearly delineates the transaction processing responsibilities of the source and target shards: all unprocessed old transactions (TX_{old}) before migration begins are handled in the source shard, while all new arriving transactions TX_{new} are redirected from the source shard to the target shard for processing. Furthermore, we schedule the related 2 shards to synchronize the latest account states in a special timestamp interleaving scenarios to ensure the correct execution of related transactions while minimizing the high overhead associated with transaction locking on the system.

The second challenge is how to provide a deterministic transaction order for the source and target shards in a dynamic operating system. MVSS schedules the processing of multiple transactions across shards. Since different shards have different transaction processing speeds, there is no guarantee that transactions will be submitted in order across shards. Therefore, it is necessary to provide deterministic order between related shards to prevent transaction disorder. Traditional global transaction ordering [18] is typically coordinated by system coordinators such as clients or reference shards, which often suffer from high latency and poor real-time performance. However, MVSS needs to dynamically adjust the transaction order of each account according to the arrival of TX_{new} and synchronize the sorting results across shards in time. Thus, third-party sorting schemes cannot meet these demands. Additionally, the target shard lacks relevant information about TX_{old} . Therefore, we designate the source shard as the sorter, which possesses real-time transaction information. The source shard continuously collects timestamp information from related transactions, updates the order list in real-time, and synchronizes the sorting results to the target shard, achieving deterministic ordering between the shards.

E. Main Contributions

The contributions of this paper are summarized as follows:

- We propose MVSS, the first cross-shard account migration solution based on multi-version state synchroniza-

tion, which reduces transaction locking while ensuring sequential execution.

- MVSS innovatively schedules related shards to process transactions cooperatively, alleviating the transaction locking issue. Additionally, we provide fine-grained order and multi-version state synchronization between relevant shards, addressing the transaction disorder problem. This paper also explores the security of MVSS, presenting solutions for various attacks.
- We implemented MVSS based on a blockchain sharding simulator and assessed its performance. Experimental results indicate that MVSS reduces locking time by 23.4%, increases throughput by 2.3%, decreases transaction latency by 7.5%, and ensures sequential execution of transactions.

The rest of this paper is organized as follows: Section II introduces the preliminaries, Section III details the MVSS mechanism, Section IV analyses the property, Section V discusses the experimental setup and results, Section VI reviews related work, and Section VII concludes the paper.

II. PRELIMINARIES

Similar to Brokerchain [9], MVSS is based on the blockchain state-sharding system which consists of multiple P2P nodes. These P2P nodes need to solve a hash puzzle to be approved to join the blockchain sharding system, which is necessary to defend against Sybil attacks [22]. Then these nodes are divided into several shards and only maintain a subset of the system's overall state data. The cuckoo rule [3] is used to defend against the join-leave attack launched by P2P nodes, which effectively protects the security of the system. In addition, MVSS is based on the account-balance blockchain transaction model, so the ledger that each shard needs to maintain during the operation of the system is mainly about the data of accounts and their balances. Within each shard, the primary node selects a certain number of transactions in the transaction pool, packs them into a block, and then forwards the block to other replica nodes in the shard. These replica nodes modify the balance information of the corresponding account according to different transactions after reaching Practical Byzantine Fault Tolerance (PBFT) consensus. Finally, the node links this block to the local blockchain as an immutable and traceable transaction record. If some cross-shard transactions cannot be processed in one shard, we use the Relay [5] mechanism to resolve them across multiple shards.

III. SYSTEM DESIGN

A. Overview of MVSS

There are two types of shards in MVSS: reference shard (R-shard) and working shard (W-shard). As shown in Fig. 1, the system comprises one R-shard and several W-shards. The responsibilities of these two type of shards in MVSS are described below:

R-shard. The R-shard is responsible for decision-making regarding account migration. R-shard nodes collect transaction information from W-shards and periodically run load-balancing algorithms, *i.e.*, Metis, CLPA, TxAllo to adjust the account distribution across W-shards. Subsequently, state blocks are generated and all W-shards are notified of the adjustment results.

W-shard. The W-shards are responsible not only for performing account migration operations based on the state block provided by the R-shard, but also for validating and executing regular transactions during each epoch and adding the generated transaction blocks to the blockchain. The shard from which an account is migrating is referred to as the source shard, while the shard to which the account is migrating is called the target shard.

B. New Data Structures of MVSS

Similar to existing research [15], we designed three auxiliary transactions for cross-shard account migration: (i) the initial migration transaction TX_{ini} sent by the source shard, (ii) the migration acknowledgment transaction TX_{ack} sent by the target shard, and (iii) the account version synchronization transaction TX_{sync} which can be sent by both the source and target shards.

TX_{ini} . Based on the new account distribution provided by the reference shard, the source shard generates TX_{ini} for the account to be migrated and sends it to the target shard. The structure of TX_{ini} is as follows:

$$TX_{ini} = \{Target, Addr, State_{ini}, \pi_{ini}, Sync\} \quad (1)$$

Target indicates the ID of the target shard, *Addr* contains the migrating account's address, *State_{ini}* and *π_{ini}* represent the account's initial state snapshot and its Merkle proof, respectively. *Sync* indicates whether state synchronization is required. If there are TX_{old} related to the migrating account in the source shard, synchronization is necessary, and *Sync* is set to true; otherwise, it is set to false.

TX_{ack} . Once the target shard receives TX_{ini} and reaches consensus, it generates TX_{ack} and sends it to both the source shard and the reference shard to confirm the successful migration. The structure of TX_{ack} is defined as follows:

$$TX_{ack} = \{TX_{ini}, \tau_{ini}, Abort\} \quad (2)$$

τ_{ini} contains the Merkle proof of TX_{ini} stored in the target shard's block. *Abort* indicates whether the migration was successful. If TX_{ini} is verified as valid, *Abort* is set to false; otherwise, it is set to true.

TX_{sync} . If there are timestamp interleaves between multiple transactions for the same migrating account, both the source and target shards can send TX_{sync} to synchronize the account's latest version state. The structure of TX_{sync} is as follows:

$$TX_{sync} = \{State_{old}, State_{new}, \pi_{new}\} \quad (3)$$

State_{old} and *State_{new}* represent the old and new versions of the migrating account's state, respectively. And *π_{old}* and *π_{new}* provide the Merkle proofs for these states.

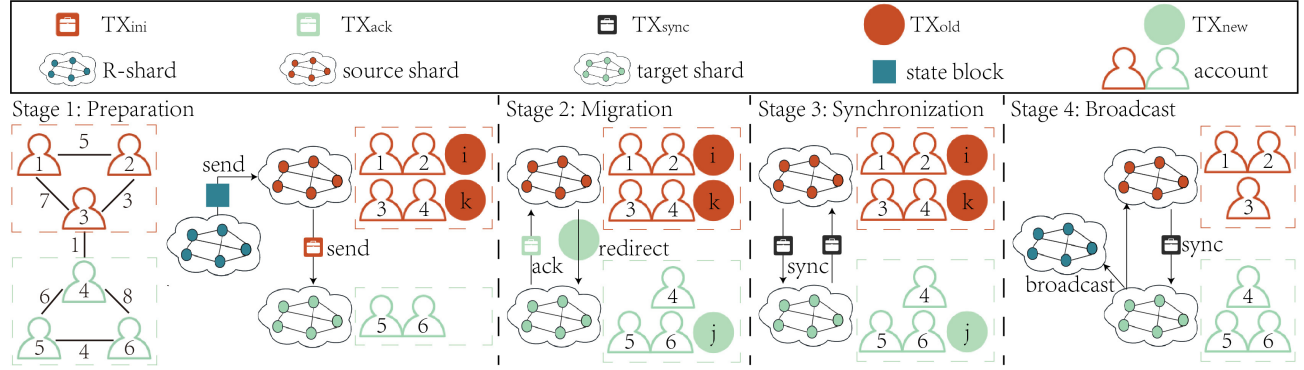


Fig. 1. An overview of MVSS.

C. Detail of MVSS

First, let us explain the scenario in which multi-version state synchronization (MVSS) needs to be executed: during the cross-shard account migration process, some timestamps of the old transactions TX_{old} may be later than the timestamps of the new transaction TX_{new} , which we refer to as timestamp interleaving. We still assume three transactions TX_i , TX_j and TX_k , where TX_i and TX_k belong to TX_{old} , while TX_j belongs to TX_{new} . The timestamps of the three transactions interleave, i.e., $TS_{old}^i < TS_{new}^j < TS_{old}^k$. In this case, to ensure that the three transactions are executed in the order of $TX_{old}^i - TX_{new}^j - TX_{old}^k$, both the source and the target shards need to synchronize the latest account states with each other after processing TX_{old}^i and TX_{new}^j , ensuring that transactions are executed according to the correct account version states.

Subsequently, we divide MVSS mechanism into 4 stages, detailed as follows:

Preparation. The source shard obtains the state block provided by the reference shard, parses the block, and retrieves the account partition results for the new epoch, identifying the accounts that need to be migrated. The source shard then records the initial state of the migrating account $State_{ini}$ and generates TX_{ini} . If there are unprocessed TX_{old} for that account in the source shard, the source shard generates an order list based on the timestamp information of transactions created by the client. After reaching consensus within the shard, TX_{ini} is sent to the target shard. During the migration process, the source shard does not process TX_{new} ; it merely records the transaction information, updates the order list, and then forwards the transaction with a redirection tag to the target shard.

Migration. Nodes in the target shard validate the effectiveness of TX_{ini} and reach consensus. If TX_{ini} is invalid, the failure handling procedure is triggered, halting the migration in the target shard, generates a TX_{ack} with *Abort* set to true, broadcasts TX_{ack} to all shards, and redirects all transactions related to the migrated account back to the source shard. If TX_{ini} is valid, the target shard creates a temporary account and ensures that the state data of that account (balance, nonce, etc.) is consistent with the initial state provided by

the source shard. If *Sync* value in TX_{ini} is false, the target shard generates TX_{ack} , sets *Abort* to false, reaches consensus within the shard, and broadcasts this to all shards, notifying that the account has been migrated to the target shard and concluding the cross-shard account migration process. If *Sync* is true, the process proceeds to the next stage.

Synchronization. The source shard continuously updates the transaction order list during the redirection process. If it detects timestamp interleaving, it temporarily holds back the submission of any TX_{old} with timestamps greater than TS_{old}^i . Once TS_{old}^i is fully submitted and the account's version state $State_{ini}$ is updated, the source shard constructs TX_{sync} to synchronize $State_{ini}$ to the target shard. The target shard receives TX_{sync} and updates the state version of the temporary account to $State_{ini}$, subsequently executing and submitting TX_{new}^j to obtain the new account version $State_j$. The target shard then constructs TX_{sync} again to synchronize $State_j$ back to the source shard. If the source shard discovers that a TX_{old} with a greater timestamp has already been submitted while redirecting TX_{new} , it will terminate that TX_{new} immediately. Once all relevant TX_{old} transactions in the source shard have been processed, the source shard constructs TX_{sync} one last time to synchronize the final version state of the account to the target shard, after which it deletes the account.

Broadcast. After the target shard updates the final state of the account, it converts the temporary account to a normal account and broadcasts message about the successful account migration to all shards, thus completing the cross-shard account migration process. Subsequently, all related transactions are handled by the target shard.

D. Defending against Attacks of MVSS

During cross-shard account migration, double-spending and replay attacks are common security threats. Below are potential scenarios for these two types of attacks, along with corresponding solutions.

Double-spending attack. A double-spending attack refers to the act of using the same funds in multiple transactions. In cross-shard account migration, the migrating account may

Algorithm 1 Multi-Version State Synchronization

```
1: for txs of the same migrated account do
2:   if timestamp interleaving is detected then
3:     wait for  $TX_{old}^i$  submitted
4:     target shard  $\leftarrow TX_{sync}$ 
5:     wait for  $TX_{new}^j$  submitted
6:     source shard  $\leftarrow TX_{sync}$ 
7:   end if
8:   if tx with the larger timestamp is submitted then
9:     abort txs arriving later with smaller timestamp
10:  else
11:    wait for all  $TX_{old}$  submitted
12:    target shard  $\leftarrow TX_{sync}$ 
13:  end if
14: end for
```

become the attacker. Assume that account α has 5 tokens before the migration begins. α initiates a transaction t_1 in the source shard, transferring 5 tokens to account β . However, when the migration starts, t_1 has not yet been processed. After migrating to the target shard, α initiates another transaction t_2 , transferring 5 tokens to account γ . Without appropriate conflict detection and handling mechanisms, α 's balance may be double spent. To address this issue, our solution involves nonce value detection by the source shard during the transaction redirection process, ensuring that each nonce value can only succeed once. If multiple transactions with the same nonce value are detected, it is deemed a double spending attack, and the source shard will terminate the transaction directly.

Replay attack. A replay attack occurs when an attacker captures a valid transaction and re-submits it for execution. During the cross-shard account migration, all nodes in the system may implement such an attack. Suppose account α initiates transaction t_1 in the source shard before migration, transferring 5 tokens to account β . If an attacker captures t_1 and replays it to the target shard after the migration begins, α 's tokens may be deducted multiple times. Our solution involves the target shard jointly checking the nonce value and the redirection tag to ensure the uniqueness of transactions. In Stage 2, if the target shard receives a transaction that lacks a redirection tag and has an abnormal nonce, it can recognize that the system has been subjected to a replay attack and will reject that transaction.

IV. ANALYSIS AND DISCUSSION

We need to ensure that MVSS has the properties of liveness, consistency, determinism and serializability during its operation. We will define and prove these properties respectively in the following. Finally, we discuss the overhead of the MVSS mechanism.

Definition 1: (Liveness.) When an account is migrated across shards, the system will run a procedure to handle whether it succeeds or fails.

Theorem 1: MVSS is able to achieve the liveness property as long as the number of malicious nodes in the source and

the target shards do not exceed the PBFT consensus security threshold.

Proof: In the migration stage of MVSS (stage 2), the target shard will verify the validity of TX_{ini} . If it is effective, it will continue to execute the subsequent program. If not, the migration failure handler is triggered, which broadcasts TX_{ack} to all shards to notify the situation. As long as the number of malicious nodes in both the source and the target shards is no more than 1/3, the honest nodes in the shard will reach a consensus on the correct result in PBFT and trigger the corresponding handle procedure. ■

Definition 2: (Consistency.) Not only the source and the target shards, but also other shards and nodes in the system can receive the final result of account migration.

Theorem 2: MVSS is able to achieve the consistency property as long as the number of malicious nodes in all shards do not exceed the PBFT consensus security threshold.

Proof: In the broadcast stage of MVSS (stage 4), the migrated account in the source shard no longer exists, and the temporary account in the target shard has been converted to the normal account, which is now fully owned by the target shard. Then the target shard broadcasts this information to all shards. As long as the number of malicious nodes in all shards is no more than 1/3, the honest nodes in the shard will reach a consensus on the correct result in PBFT. After that, all transactions related to this account are sent to and processed by the target shard. ■

Definition 3: (Determinism and serializability.) During the cross-shard account migration, the related transactions are executed and committed in a sequential order.

Theorem 3: MVSS is able to achieve the determinism and serializability properties as long as the number of malicious nodes in the source and the target shards do not exceed the PBFT consensus security threshold.

Proof: In the synchronization stage of MVSS (stage 3), The sorted results of the source shard according to the transaction timestamp need to reach consensus in the shard before they can be synchronized to the target shard through TXsync. The target shard will also verify TXsync and the sorted results and reach a consensus. As long as the number of malicious nodes in the source and the target shards is no more than 1/3, the honest nodes in the shard will reach a consensus on the correct result in PBFT. So both shards serialize transaction executions and commits in the same deterministic order. Even if the special case of timestamp interleaving is encountered, we provide suitable processing schemes to ensure that transactions are executed and committed in order. ■

Overhead discussion. To achieve the goal of scheduling the source and the target shards to process related transactions cooperatively, we allow the two relevant shards to each process a portion of related transactions during cross-shard account migration. At the same time, we need to ensure that multiple transactions related to the same account are executed in order, so we can't allow the source and the target shards to process transactions in parallel optimistically. Typically, the timestamp of TX_{old} in the source shard is smaller, so it is processed and

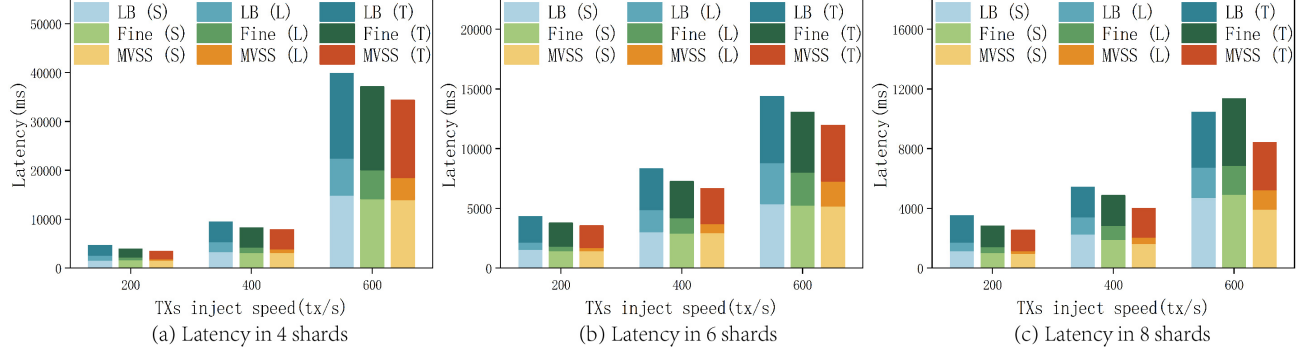


Fig. 2. Average latency breakdown with different protocols, number of shards and TXs inject speed.

committed first, and then the target shard processes TX_{new} based on the final state of the account synchronized by the source shard, with low overhead. In the special timestamp interleaving scenario, the system needs to synchronize the state version of the migrated account between the source shard and the target shard multiple times. During the state synchronization, the execution of subsequent transactions needs to be blocked to ensure the deterministic and serialization properties of MVSS, so the system overhead will increase in this case. The actual impact of overhead on system performance is discussed further in Section V.

V. PERFORMANCE EVALUATION

A. System Implementation

We implemented MVSS using over 2000 lines of *Go* based on the open-source platform BlockEmulator¹ [16], utilizing a dataset of 200,000 historical Ethereum transactions provided by the XBlock² [17]. Our experimental system is divided into four modules, which are (i) the storage module based on boltDB and levelDB; (ii) the data module containing the new auxiliary transaction data structure; (iii) the consensus module implementing the MVSS operation logic; (iv) the communication module based on TCP.

The baseline for our experiments consisted of two existing cross-shard account migration protocols: LB [14] and Fine-tuned [15], which are representatives of the full-locked and semi-locked mechanisms, respectively. The experimental variables included the number of W-shards and the speed at which transactions were injected into the system, specifically set to 200 ~ 600 transactions per second across 4 ~ 8 W-shards. Other default parameters were as follows: (i) block size of 500; (ii) block interval of 2 seconds; (iii) maximum auxiliary transactions per block set at 300; and (iv) the R-shard calling the CLPA algorithm for account reshuffling and migration after every 50,000 confirmed transactions.

B. Performance Metrics

We assessed the impact of account migration on related transactions and system performance from four perspectives:

¹Available at <https://github.com/HuangLab-SYSU/block-emulator>

²Available at <https://www.xblock.pro>

(i) **Latency.** Latency breakdown is a measure of the time consumed by related transactions at different stages during cross-shard account migration. We divided it into three parts, which include the queuing latency in the source shard (S), the transaction locking latency (L), and the queuing latency in the target shard (T):

$$Latency = S + L + T(ms) \quad (4)$$

(ii) **Ratio of locked TXs (RLT).** RLT is used to observe the probability that a related transaction will be locked during a cross-shard account migration. We defined it as the number of locked transactions (TX_{lock}) divided by the difference between the total number of regular transactions (TX_{reg}) and auxiliary transactions (TX_{aux}):

$$RLT = \frac{\sum TX_{lock}}{\sum TX_{reg} + \sum TX_{aux}} \quad (5)$$

(iii) **Throughput (TPS).** TPS is an important metric to measure the transaction processing capability of a system, and we define TPS as the amount of TX_{reg} processed by the system during account migration divided by the account migration time (represented by t_{am}):

$$TPS = \frac{\sum TX_{reg}}{t_{am}}(txs/s) \quad (6)$$

(iv) **Ratio of disorderly transactions (RDT).** RDT is used to observe the chance that related transactions will be submitted in an incorrect order. We defined it as the number of transactions experiencing timestamp interleaving (TX_{ti}) issues divided by the total number of TX_{reg} :

$$RDT = \frac{\sum TX_{ti}}{\sum TX_{reg}} \quad (7)$$

C. System Evaluation and Analysis

Hereinafter, we will respectively analyze the performance of MVSS in 4 experimental metrics.

Latency: As shown in Fig. 2(a-c), as the TX inject speed increased from 200 txs/s to 600 txs/s, transaction latency progressively rose with the increasing system load. Compared to state-of-the-art 2 locking schemes, MVSS demonstrated significant advantages in both total transaction latency and

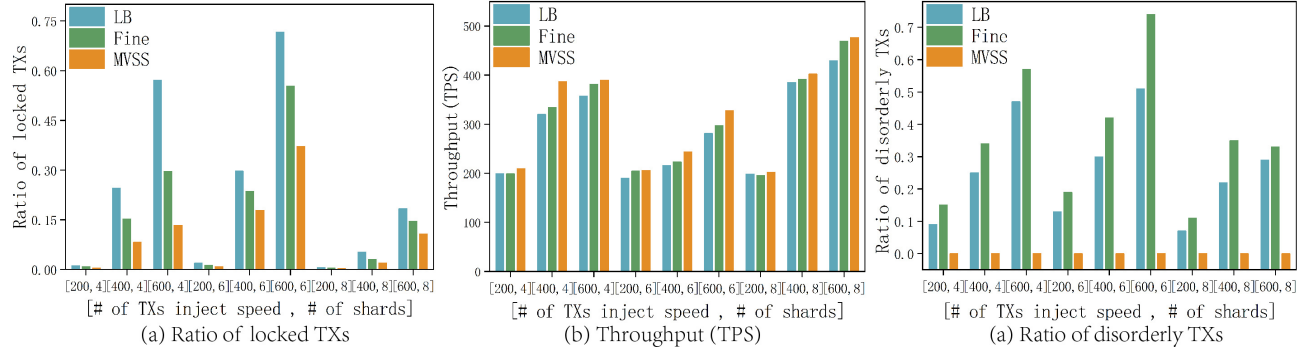


Fig. 3. Ratio of locked TXs, Throughput and Ratio of disorderly TXs with different protocols, number of shards and TXs inject speed.

transaction locking time. In Fig. 2(a), when the TX inject speed reached 600 txs/s, MVSS's total transaction latency and locking time were reduced by 7.5% and 23.4%, respectively, compared to the state-of-the-art study. The trends in Fig. 2(b) and Fig. 2(c) are similar.

Ratio of locked TXs: As illustrated in Fig. 3(a), the traditional full-lock scheme, such as LB, already has more than 70% locked proportion of transactions related to the migration account in the scenario of high system load. However, MVSS still outperformed other protocols in terms of the ratio of locked transactions, achieving a minimum locking rate of only 0.3% in some scenarios. A small number of locked transactions is necessary to address the timestamp interleaving issue, as MVSS ensures that transactions are executed in order during account migration.

Throughput: From Fig. 3(b), we observe that system throughput during account migration rises as TX inject speed increases. Compared to the baselines, MVSS exhibits superior performance in throughput. When the number of W-shards is 4 and TX inject speed reaches 600 txs/s, MVSS's throughput improves by 9.1% and 2.3%, respectively. MVSS employs a multi-version state synchronization strategy to reduce transaction locking, which may slightly increase the makespan of some transactions; however, experimental results indicate that it enhances the handling capacity of blocked transactions in the transaction pool. The significant improvement in transaction processing capability during cross-shard account migration compensates for the performance loss.

Ratio of disorderly TXs: As shown in Fig. 3(c), only MVSS can ensure sequential execution, whereas the other two schemes exhibit transaction disorder. When TX inject speed reaches 600 txs/s, over 70% of accounts cannot guarantee transaction order execution and final state correctness. The main reasons for this phenomenon are (i) As blocked transactions accumulate in the transaction pool, the number of transactions related to a single migrated account increases, raising the probability of timestamp disorder; (ii) The processing speeds of intra-shard and cross-shard transactions differ, and processing cross-shard transactions via Relay [5] mechanism takes longer, causing later transactions to be executed earlier. MVSS ensures sequential execution by providing fine-grained

ordering and multi-version state synchronization.

VI. RELATED WORK

Blockchain [19], [20] has attracted a lot of attention due to its decentralization. Sharding is a technology used to address blockchain scalability issues. It is mainly divided into two categories: network-sharding and state-sharding. The network sharding, represented by Elastico[1], which applies the concept of database system sharding to blockchain and significantly improves the parallel processing capability of blockchain. Subsequently, Omniledger [2] designed ByzCoinX and Atomix as intra-shard and cross-shard consensus, respectively, to further improve the system transaction throughput. Meanwhile, Rapidchain [3] proposes a reconfiguration scheme based on the Cuckoo rule to reduce the sharding overhead. Then, Chainspace [4] proposes an inter-shard atomic commit protocol named S-BAC for the efficient processing of smart contracts. Monoxide [5] proposes a cross-shard processing mechanism called Relay. And [6] proposed an on-chip consensus protocol called AHL. Recently, the following types of protocols proposed state sharding, which allows each shard to store only a portion of the blockchain's transaction history and ledger, thereby reducing the storage burden on nodes. Pyramid [7] has proposed a novel layered state-sharding scheme. MVCom further optimizes shard reconfiguration by scheduling the most valuable shard committees

In state-sharding blockchain based on account-balance model, cross-shard transactions and inter-shard load balancing are very difficult. Account shuffling is one of the most popular solutions, which falls into two main categories. Brokerchain [9] is representative of the first category of schemes, which require the establishment of separate reconfiguration phases during system epochs for account redistribution. In addition, the Constrained Label Propagation Algorithm (CLPA) optimizes the pattern of account allocation between shards. TxAllo combines global and adaptive community detection algorithms to further optimize the account shuffling scheme. However, these solutions disrupt real-time transaction processing in blockchain sharding systems and increases transaction delays, thereby affecting overall system performance. Schemes represented by [14], [15] propose to perform cross-shard account

migration without stopping processing regular transactions. Experimental results show that these schemes have higher system throughput.

VII. CONCLUSION

This paper proposes MVSS, the first blockchain cross-shard account migration mechanism based on multi-version state synchronization. MVSS innovatively schedules the source shard and the target shard to process related transactions cooperatively, which mitigates transaction locking issues. Additionally, we employ multi-version state synchronization across shards based on fine-grained order to address transaction disorder problems. Experimental results show that, compared to state-of-the-art studies such as Fine-tuned scheme, MVSS not only reduces transaction latency by 7.5% but also ensures the sequential execution of cross-shard transactions, further providing a reference for achieving inter-shard account shuffling and load balancing.

VIII. ACKNOWLEDGEMENT

This work is supported by the National Key R&D Program of China under Grant No. 2022YFB2702100.

REFERENCES

- [1] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A Secure Sharding Protocol For Open Blockchains," in *Proc. of ACM CCS*, 2016, pp. 17–30.
- [2] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *Proc. of IEEE S&P*, 2018, pp. 583–598.
- [3] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proc. of ACM CCS*, 2018, pp. 931–948.
- [4] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," in *Proc. of NDSS*, 2018.
- [5] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proc. of USENIX NDSI*, 2019, pp. 95–112.
- [6] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. of ACM SIGMOD*, 2019, pp. 123–140.
- [7] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proc. of IEEE INFOCOM*, 2021, pp. 1–10.
- [8] H. Huang, Z. Huang, X. Peng, Z. Zheng, and S. Guo, "MVCOM: Scheduling Most Valuable Committees for the Large-Scale Sharded Blockchain," in *Proc. of IEEE ICDCS*, 2021, pp. 629–639.
- [9] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. of IEEE INFOCOM*, 2022, pp. 1968–1977.
- [10] C. Li, H. Huang, Y. Zhao, X. Peng, R. Yang, Z. Zheng, and S. Guo, "Achieving scalability and load balance across blockchain shards for state sharding," in *Proc. of IEEE SRDS*, 2022, pp. 284–294.
- [11] Y. Zhang, S. Pan, and J. Yu, "TxAllo: Dynamic Transaction Allocation in Sharded Blockchain Systems," in *Proc. of IEEE ICDE*, 2023, pp. 721–733.
- [12] K. Mu and X. Wei, "EfShard: Toward Efficient State Sharding Blockchain via Flexible and Timely State Allocation," *IEEE Transactions on Network and Service Management*, 2023, vol. 20, no. 3, pp. 2817–2829.
- [13] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière, "Shard scheduler: object placement and migration in sharded accountbased blockchains," in *Proc. of ACM AFT*, 2021, pp. 43–56.
- [14] M. Li, W. Wang, and J. Zhang, "LB-Chain: Load-Balanced and Low-Latency Blockchain Sharding via Account Migration," *IEEE Transaction on Parallel Distributed System*, 2023, vol.34, no.10, pp. 2797–2810.
- [15] H. Huang, Y. Lin, and Z. Zheng, "Account Migration across Blockchain Shards using Fine-tuned Lock Mechanism," in *Proc. of IEEE INFOCOM*, 2024, pp. 271–280.
- [16] H. Huang, G. Ye, Q. Chen, Z. Yin, X. Luo, J. Lin et al., "BlockEmulator: An Emulator Enabling to Test Blockchain Sharding Protocols," 2023, arXiv: arXiv:2311.03612. [Online]. Available: <http://arxiv.org/abs/2311.03612>
- [17] P. Zheng, Z. Zheng, J. Wu, and H.-N. Dai, "XBlock-ETH: Extracting and Exploring Blockchain Data From Ethereum," *IEEE Open Journal of the Computer Society*, 2020, vol. 1, pp. 95–106.
- [18] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang et al., "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," in *Proc. of IEEE INFOCOM*, 2023, pp. 1–10.
- [19] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Tech. Rep.*, 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [20] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, 2014, [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [21] M. Castro, B. Liskov et al., "Practical byzantine fault tolerance," in *Proc. of OSDI*, 1999, vol. 99, no. 1999, pp. 173–186.
- [22] S. Zhang and J.-H. Lee, "Double-spending with a sybil attack in the bitcoin decentralized network," *IEEE Transactions on Industrial Informatics*, 2019, vol. 15, no. 10, pp. 5715–5722.